Searching

23B. Searching Algorithms (8 problems)

Pattern recognition: "Find target in sorted data", "First/last occurrence", "Minimum/maximum that satisfies condition"

23.13 Linear Search

Approach: Scan each element left to right and return the first index whose value equals the target, or -1 if none match.

JAVASCRIPT Copy

```
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;
  }
  return -1;
}
```

O(n), use when data is unsorted. Iterate and compare each element.

23.14 Binary Search

JAVASCRIPT Copy

```
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}
```

Time: O(log n). **Space:** O(1).

Frontend connection: Used in virtualized lists — given a scroll position, binary search finds the visible item index.

23.15 Binary Search on Answer

"Find the minimum/maximum value that satisfies a condition." Search a range of possible answers, not an array.

JAVASCRIPT Copy

```
function shipWithinDays(weights, days) {
  let lo = Math.max(...weights), hi = weights.reduce((a, b) => a + b);
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (canShip(weights, days, mid)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}

function canShip(weights, days, capacity) {
  let daysNeeded = 1, currentLoad = 0;
  for (const w of weights) {
    if (currentLoad + w > capacity) { daysNeeded++; currentLoad = 0; }
    currentLoad += w;
  }
```

23.16 Search in Rotated Sorted Array

JAVASCRIPT Copy

```
function search(nums, target) {
  let lo = 0, hi = nums.length - 1;
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] ═══ target) return mid;
    if (nums[lo] ≤ nums[mid]) {
      if (target ≥ nums[lo] && target < nums[mid]) hi = mid - 1;
      else lo = mid + 1;
    } else {
      if (target > nums[mid] && target ≤ nums[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}
```

23.17 First and Last Occurrence

JAVASCRIPT Copy

```
function findFirst(arr, target) {
  let lo = 0, hi = arr.length - 1, result = -1;
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] ═══ target) { result = mid; hi = mid - 1; }
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return result;
```

```
function findLast(arr, target) {
  let lo = 0, hi = arr.length - 1, result = -1;
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] === target) { result = mid; lo = mid + 1; }
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return result;
}
```

23.18 Search in 2D Matrix

Approach: Treat the row-sorted matrix as one flat sorted array and binary search it, mapping each mid index back to row/column via divide and modulo by the column count.

JAVASCRIPT Copy

```
function searchMatrix(matrix, target) {
  if (!matrix.length || !matrix[0].length) return false;
  const rows = matrix.length, cols = matrix[0].length;
  let lo = 0, hi = rows * cols - 1;
  while (lo ≤ hi) {
    const mid = (lo + hi) >> 1;
    const val = matrix[Math.floor(mid / cols)][mid % cols];
    if (val === target) return true;
    if (val < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return false;
}
```

Treat as flat sorted array. `row = mid / cols`, `col = mid % cols`. Standard binary search.

then binary search within that bounded window.

JAVASCRIPT Copy

```
function exponentialSearch(arr, target) {
  const n = arr.length;
  if (n === 0) return -1;
  if (arr[0] === target) return 0;
  let bound = 1;
  while (bound < n && arr[bound] < target) bound *= 2;
  let lo = Math.floor(bound / 2);
  let hi = Math.min(bound, n - 1);
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}
```

Find range (double: 1, 2, 4, 8...), then binary search within. Useful for unbounded/infinite arrays.

23.20 Interpolation Search

Approach: Estimate the probe position by linearly interpolating where the target should lie between the current low and high values, then narrow the range like binary search.

JAVASCRIPT Copy

```
function interpolationSearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi && target >= arr[lo] && target <= arr[hi]) {
    if (lo === hi) return arr[lo] === target ? lo : -1;
    const pos = lo + Math.floor(
```

```
    if (arr[pos] < target) lo = pos + 1;
    else hi = pos - 1;
  }
  return -1;
}
```

Estimates position based on value distribution. O(log log n) for uniform data, O(n) worst.

QUICK CHECK — no marks, just for you.

1. The lesson's binarySearch is run against a sorted array, once for a value that is present and once for a value that is absent. What does this print?

JAVASCRIPT Copy

```
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}

console.log(binarySearch([1, 3, 5, 7, 9, 11], 11));
console.log(binarySearch([1, 3, 5, 7, 9, 11], 4));
```

- ○ 5
5
- ○ -1
-1
- ○ 5
-1

Check answer

2. The lesson's rotated-sorted-array search is called on a rotated array for a value that exists and a value that does not. What does this print?

JAVASCRIPT Copy

```javascript
function search(nums, target) {
  let lo = 0, hi = nums.length - 1;
  while (lo <= hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] === target) return mid;
    if (nums[lo] <= nums[mid]) {
      if (target >= nums[lo] && target < nums[mid]) hi = mid - 1;
      else lo = mid + 1;
    } else {
      if (target > nums[mid] && target <= nums[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}

const rotated = [4, 5, 6, 7, 0, 1, 2];
console.log(search(rotated, 0));
console.log(search(rotated, 3));
```

- ○ -1
-1
- ○ 4
3
- ○ 0
-1
- ○ 4
-1

3. In the lesson's shipWithinDays solution, which uses the "Binary Search on Answer" pattern, what is the search actually being performed over?

- ○ The list of day counts from 1 up to days, checking each for feasibility
- ○ The indices of the weights array, narrowed until the target index is found
- ○ The range of candidate ship capacities, from the largest single weight up to the sum of all weights
- ○ The weights array itself, which must be sorted before the search begins

Check answer

‹ Previous
Sorting algorithms

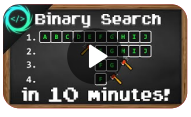
Next ›
Two pointer

LEARN MORE

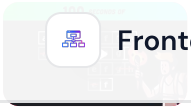
ARTICLES & DOCS

Binary search (GeeksforGeeks)
geeksforgeeks.org

VIDEOS



Learn binary search in 10 minutes
YouTube



Frontend System Desi...

YouTube



Binary search explained visually

YouTube

Hand-picked references for this lesson.